Semantic tagging

MCP API Examples · built Nov 13, 2026

MCP API Examples

> ⬅️ Back to MCP Server tab | Setup: VS Code Copilot | Setup: Claude Desktop

Complete reference of natural-language prompts for 50+ MCP tools across 16 categories. Copy any prompt and paste into Copilot Chat or Claude — the LLM automatically maps your question to the right tool and parameters.

Haystack Tagging 7 tools

`t3000_haystack_list_tags` — Discover available tags

Discover the full Haystack tag vocabulary. Tags are organized by category: `haystack` (standard v4 tags like `air`, `sensor`, `temp`), `brick` (Brick ontology classes), and `custom` (user-defined). Returns tag names, descriptions, parent relationships, and how many points use each tag.

What Haystack tags are available?

Show me all Haystack tags in the brick category

List custom tags defined in the system

`t3000_haystack_get_point_tags` — Get tags assigned to points

Retrieve all Haystack tags assigned to specific device points. Filter by device serial number and optionally by point type (INPUT, OUTPUT, VARIABLE). Returns a flat list of point→tag assignments showing what semantic meaning each point carries.

What tags are assigned to device 233626?

Get tags for the INPUT points on device 237219

Show me all tags for OUTPUT points on devices 233626 and 240488

`t3000_haystack_search_points` — Find points by tags

Search across all devices for points that have specific combinations of Haystack tags. Points must match ALL specified tags. Use this to find all temperature sensors, all outside air points, all command points, etc. Optionally restrict by device serials or point types.

Search for all temperature sensors

Find outside air temperature sensors

Show me all humidity sensors on device 237219

`t3000_haystack_auto_tag` — Run auto-tagging

Apply the auto-tagging engine to devices. First applies range-based rules (digital/analog, engineering units, value ranges), then regex rules that match point labels. Assigns Haystack tags AND Brick ontology classes to every point. Returns the count of points tagged.

Auto-tag device 233626

Auto-tag all my devices: 233626, 237219, 240488

`t3000_haystack_preview_tags` — Preview tags without saving

See exactly what tags and Brick classes would be assigned by auto-tagging BEFORE writing to the database. Essential for testing new rules or verifying tag assignments. Returns full preview with tags, Brick class, and the rule that matched for each point.

Preview what tags would be assigned to device 240488

Show me a preview of auto-tagging for device 233626 without applying

`t3000_haystack_list_rules` — Show all tagging rules

Lists every auto-tagging rule in the system with its regex pattern, target tags, Brick class (if any), category, priority, and enabled/disabled status. Currently 248 rules covering Haystack v4 tags and Brick ontology classes.

List the Haystack auto-tagging rules

Show me all enabled tagging rules in the brick category

`t3000_haystack_get_brick_class` — Get Brick ontology classes

Brick is a formal ontology for buildings (brickschema.org). After auto-tagging, each point gets a Brick class like `Supply_Air_Temperature_Sensor` or `Outside_Air_Flow_Sensor`. Use this to check what Brick classes are assigned to your device points.

What Brick class does input 8 on device 237219 have?

Show all Brick classes for device 233626

Data & Discovery 7 tools

`t3000_device_list` — Enumerate all devices

Get every T3000 device in the system with serial number, name, device type, building/floor/room, and counts of INPUT, OUTPUT, and VARIABLE points. Optionally filter by device name substring or refresh the device list to discover new devices on the network.

List all T3000 devices

Find devices named T3-NB-ESP

How many devices are in this system?

Refresh device list to discover new devices

`t3000_device_get` — Get a single device record

Query one device by serial number and return its full record: serial, name, product type, IP address, building/floor/room, online/offline status, panel and network info, and timestamps. Use this when you need a specific device's details rather than the whole list.

Get the full details for device 233626

What is the IP address of device 240488?

Is device 237219 online?

`t3000_scan_network` — Scan network for T3000 devices

Scan the local network for T3000 devices via UDP broadcast (0x64/0x65 protocol). Returns discovered devices with serial numbers, IP addresses, product types, firmware versions, panel names, and subnet info. Takes 5–10 seconds. Use this to discover new or replacement devices on the network.

Scan the network for T3000 devices

Discover any new T3000 panels on the network

Find all T3000 devices on my LAN

`t3000_device_get_points` — Get all points for a device

Returns every point on a device — labels, engineering units, Haystack tags, and Brick classes. Optionally filter by point type to get only INPUTs, OUTPUTs, or VARIABLEs.

Show me the input points for device T3-NB-ESP

Show all VARIABLE points on device 233626

List all points on device 240488

`t3000_point_get_metadata` — Full metadata for one point

Deep-dive into a single point: label, full label, engineering units, range (low/high), description, digital vs analog, all Haystack tags, and Brick class. Useful when you need to understand exactly what a point represents.

Get full metadata for input 0 on device 240488

What are the units and range for output 3 on device 237219?

`t3000_metadata_search` — Cross-device label search

Search point labels across all devices by keyword. Matches against point labels, full labels, and descriptions. Optionally filter by device serials and point types. Great for finding all points related to "temperature", "flow", "pressure", etc.

Search for points labeled temperature

Search for flow across all output points

Find all points with fan in the name on device 233626

`t3000_point_search` — Semantic point search

Search points across devices using natural language. Matches against point labels, Haystack tags, Brick classes, and descriptions. Returns ranked results by relevance. Much smarter than label search — "supply air temperature" matches `Supply_Air_Temperature_Sensor` even if the label is abbreviated.

Find the temperature sensor in the lobby

Show me all fan speed outputs

Search for discharge air temperature on device 233626

Operational — Read/Write 5 tools

`t3000_point_read` — Read a single point value

Read the current (last synced) value of any point from the database. Returns the value, engineering units, label, and timestamp of the last reading.

Read input point 0 on device 233626

What's the current value of output 3 on device 237219?

Read variable 12 on device 240488

`t3000_point_write` — Write to any point field

Write to any writable field on a point. Defaults to `value` (fValue). Also supports: `label`, `description`, `range`, `auto_manual`, `digital_analog`. All other fields are preserved from the current device state. **Safety:** requires `confirm: true` for OUTPUT and VARIABLE points. Writes go through the C++ FFI layer to the actual device.

Set output 5 on device 233626 to 72.5

Turn on output 7 on device 237219

Rename input 2 on device 240488 to "ZoneTemp"

Change description of output 3 to "Main Fan Speed"

Set the range of variable 0 on device 233626 to 100

`t3000_point_read_batch` — Read multiple points at once

Read values for multiple points in a single call. Points can span different devices and point types. Much faster than calling `t3000_point_read` repeatedly for bulk operations.

Read inputs 0, 1, and 2 on device 240488 all at once

Read input 0 on 233626 and output 0 on 237219 together

`t3000_point_write_batch` — Write multiple points at once

Write values to multiple points in a single call. Each point can specify an optional `field` (defaults to `value`). Failures are reported per-point; partial success is supported. Requires `confirm: true`. Points can span different devices.

Set outputs 0 through 3 on device 237219 to 100

Set output 1 to 80 and rename it to "PumpSpeed"

`t3000_point_batch_metadata` — Get full metadata for multiple points

Get complete metadata for multiple points in a single call. Returns label, engineering units, range, digital/analog type, description, current value, Haystack tags, and Brick class for each point. Much more efficient than calling `t3000_point_get_metadata` N times — use when you need full context on several points at once.

Give me full metadata for inputs 0 through 4 on device 240488

What are the tags, units, and ranges for output 0 of device 233626 and input 1 of device 237219?

Device Operations 8 tools

`t3000_trendlog_list` — Discover available trendlogs

List all trendlogs configured for a device. Returns trendlog IDs, labels, logging interval, buffer size, and how many points each trendlog tracks. Use this to discover what historical data is available before querying with `t3000_trendlog_query`.

What trendlogs are available on device 240488?

List all logging configurations for device 233626

`t3000_trendlog_export` — Export trendlog history as CSV/JSON

Export all historical data from a trendlog in one call. Queries every point in the trendlog and returns timestamped values. Use after `t3000_trendlog_list` to discover trendlog IDs. Defaults to CSV format with columns: timestamp, point_type, point_index, point_id, value, units, range, digital_analog.

Export trendlog 1 from device 240488 as CSV for the last 24 hours

Export all trendlog data from device 233626 trendlog 0 as JSON since 2026-01-01

`t3000_device_refresh` — Force refresh from hardware

Refresh point data directly from the physical device via the C++ FFI layer (Action 17). Updates the database with the latest values from the hardware. Optionally filter by point type to refresh only inputs, outputs, or variables.

Refresh all data from device 240488

Refresh only the inputs on device 233626

`t3000_schedule_list` — List device schedules

List all schedules configured on a device. Returns schedule IDs, daily time settings for each day of the week (Monday–Friday), assigned outputs/variables, holiday settings, and interval configuration.

Show me the schedules on device 240488

What time-based automation is configured on device 233626?

`t3000_settings_read` — Read device settings

Read all device configuration from the database. Returns 8 categories: network (IP/subnet/gateway/DHCP), communication (COM ports/baudrates/parity/stopbits), time (timezone/NTP/DST), protocol (Modbus ID/MSTP/BACnet), DynDNS, hardware info, feature flags, and email settings. Optionally filter to a single category.

Read all settings for device 240488

Show me the network configuration of device 233626

What are the communication settings for device 240488?

`t3000_settings_write` — Update device settings

Update device configuration fields. Supports network (ip_address, subnet, gateway, tcp_type), communication (com0/1/2_config, com_baudrate0/1/2), time (time_zone, enable_sntp, sntp_server), and email (smtp_server, email_address, etc.). Requires confirm:true.

Set device 240488 to use DHCP

Change the NTP server on device 233626 to pool.ntp.org

Configure COM1 on device 240488 as Modbus Slave at 19200 baud

`t3000_device_control` — Reboot or factory reset

Send control commands to a device: reboot (restart the controller) or reset_defaults (factory reset to default settings). Requires confirm:true for safety.

Reboot device 240488

Factory reset device 233626

`t3000_device_delete` — Delete a device record

Remove a device and its database record by serial number. Destructive and irreversible — requires confirm:true. Only the local T3000 database record is removed; the physical controller on the network is unaffected. Use `t3000_device_list` first to confirm the serial number before deleting.

Delete device 233626 from the project

Remove device 271845 from the database

Control Logic 5 tools

`t3000_program_list` — List PLC programs

List all control logic programs running on a device. Returns program IDs, labels, status (running/stopped), auto/manual mode, program size, and switch node. Use before `t3000_program_read` to find which programs exist.

List all programs on device 240488

What control logic is running on device 233626?

`t3000_program_read` — Read program source code

Read a specific PLC program's full details including source code (truncated to 2000 chars), label, status, auto/manual mode, size, and switch node. The full source length is included in the response.

Show me the source code of program 1 on device 240488

What does program 'MAIN_LOOP' on device 233626 do?

`t3000_pid_list` — List PID control loops

List all PID control loops on a device. Returns loop IDs, current setpoint, process variable (input value), output value, P/I/D tuning parameters (proportional, reset, rate), bias, action type, auto/manual mode, setpoint limits, and status. Essential for HVAC diagnostics.

List all PID loops on device 240488

Show me the tuning parameters for the PID loops on device 233626

Which PID loops are in manual mode?

`t3000_holiday_list` — List holiday exceptions

List all holiday schedule exceptions on a device. Returns holiday IDs, dates (month/day/year), holiday output values, auto/manual mode, and status. Holidays override the normal weekly schedule on their

designated dates.

List holiday schedules on device 240488

What holiday exceptions are configured on device 233626?

`t3000_building_summary` — System-wide overview

Get a one-shot dashboard of the entire building automation system. Returns total device count with names, active alarm count, total trendlogs, schedules, programs, and PID loops across all devices. Includes a health indicator (good/warning/critical based on alarm count). Perfect for "How's the building?" queries.

How is the building doing right now?

Give me a system overview

Are there any active alarms in the system?

Diagnostics 2 tools

`t3000_device_diagnostics` — Single device health check

Run a comprehensive diagnostic check on a single device. Returns: device name, firmware version, hardware revision, IP address, point counts (inputs/outputs/variables), active alarm count, trendlog count, program status (total/running), schedule count, PID loop status (total/in-auto), and an overall health rating (good/warning/needs_attention).

Run diagnostics on device 233626

Check the health of device 240488

`t3000_device_diagnostics_batch` — Multi-device health check

Run diagnostics on multiple devices or all devices at once. Returns per-device health reports plus an overall building health summary (good/warning/needs_attention). Pass no serial_numbers to diagnose everything. Use for system-wide health assessments.

Check the health of all devices

Run diagnostics on devices 233626, 237219, and 240488

Config & Users 3 tools

`t3000_alarm_settings_read` — Read alarm thresholds

Read alarm threshold configuration for a device. Returns alarm rules: which points are monitored, the comparison condition, low/high/normal/way-low/way-high threshold values, and time delays. This is alarm configuration — use `t3000_alarm_list` for active alarms.

Read alarm threshold settings for device 240488

What alarm rules are configured on device 233626?

`t3000_users_list` — List device users

List all users configured on a device. Returns user IDs, names, access levels (View/Full/Graphic/Routine), rights, default panel/group assignments, and status.

List all users on device 240488

Who has access to device 233626?

`t3000_graphics_list` — List graphic screens

List all graphic/HMI screens available on a device. Returns graphic IDs, labels, descriptions, picture files, total points per screen, and switch nodes.

List graphic screens on device 240488

What HMI screens are available on device 233626?

Documentation 2 tools

`t3000_doc_list` — List documentation topics

List all T3000 documentation topics organized by section: Quick Start, Architecture, Device Management, Data Points, Features, API Reference, Guides, Building Platform, and Haystack & MCP. Use to discover what docs exist before reading a specific one.

What documentation is available?

Show me the API reference docs

`t3000_doc_read` — Read a documentation page

Read the full markdown content of a T3000 documentation page. Pass the path from `t3000_doc_list` (e.g. `quick-start/overview` or `haystack/mcp-api-examples`). Fetches from local filesystem in dev mode, falls back to GitHub raw in production.

Show me the MCP API examples documentation

Read the quick start guide

What does the troubleshooting guide say about connection issues?

Core 3 tools

`t3000_ping` — Health check

Simple connectivity test. Returns server status, current timestamp, and server name.

Is the T3000 MCP server running?

Ping the MCP server

`t3000_get_version` — Server metadata

Returns server name, version number, MCP protocol version (2025-03-26), and total tool count.

What version is the MCP server?

`t3000_describe_tool` — Tool documentation

Get the complete input schema, description, and parameter details for any tool by name. Useful for LLM agents to understand tool capabilities before calling.

Describe the device_list tool

What parameters does trendlog_query accept?

Analytics & Export 2 tools

`t3000_haystack_validate` — Validate tagging quality

Run ontology validation rules against tagged points. Checks for: sensor tags on non-INPUT points, command tags on non-OUTPUT points, missing required tags, conflicting tag combinations, invalid Brick class assignments, and orphaned tag references. Returns warnings and errors.

Validate the Haystack tags on device 237219

Validate tagging across all devices

Are there any tagging errors in device 233626?

`t3000_haystack_export` — Export semantic model

Export the full semantic model for devices in standard formats. **haystack-json**: Project Haystack tagged entity format. **brick-ttl**: Brick ontology in Turtle RDF (W3C standard). **brick-jsonld**: Brick ontology in JSON-LD (linked data). **csv-flat**: Flat CSV table of all points with serial_number, point_type, point_index, label, description, current value, units, range, digital/analog, Haystack tags, and Brick class. Use for integration with other building systems or semantic analysis tools.

Export device 233626 as Brick Turtle RDF

Export all three devices as Haystack JSON

Export device 240488 as Brick JSON-LD

Export device 233626 as a flat CSV spreadsheet

Rules Management 2 tools

`t3000_rule_toggle` — Enable or disable rules

Toggle an auto-tagging rule on or off by its rule ID. Disabled rules are skipped during auto-tagging but remain in the database. Use `t3000_haystack_list_rules` to find rule IDs.

Disable auto-tagging rule 5

Enable rule 3

`t3000_rule_create` — Create a new tagging rule

Add a custom auto-tagging rule. Specify a regex pattern to match point labels, the Haystack tags to assign, and optionally a Brick class. Rules can target specific point types, engineering units, or object types. Higher priority rules are applied first.

Create a rule that tags any point with CO2 in the label as air, co2, sensor

Add a Brick rule to classify ZoneTemp labels as Zone_Air_Temperature_Sensor

---

Alarms & Trends 3 tools

`t3000_alarm_list` — List alarms

Get all alarms in the system with severity, message, timestamp, and acknowledgment status. Filter by device serials or show only unacknowledged (active) alarms.

List all active alarms

Show me alarms for device 233626

Are there any unacknowledged alarms on device 237219?

`t3000_alarm_acknowledge` — Acknowledge an alarm

Acknowledge a specific alarm by device serial number and alarm ID. Acknowledged alarms are marked with a timestamp. Use `t3000_alarm_list` to find alarm IDs.

Acknowledge alarm abc123 on device 233626

`t3000_trendlog_query` — Query historical trend data

Retrieve time-series data for any point over a specified time range. Specify start and optional end time in ISO 8601 format, and limit the number of data points returned. Returns timestamp-value pairs with engineering units and point metadata.

Get trend data for input 8 on device 237219

Show me trend data for variable 0 on device

for the last hour

240488 from yesterday to now

Get the last 100 readings for output 2 on device 233626

Task Management 4 tools

`t3000_task_create` — Create a workflow task

Create a new task for tracking commissioning, maintenance, or troubleshooting workflows. Tasks have a title, description, status (pending/in_progress/completed), priority (low/normal/high/critical), and optional device reference.

Create a task: configure AHU-1 network settings

Add a high-priority task to troubleshoot device 233626 alarms

`t3000_task_list` — List tasks

List all tasks with optional filters by status (pending, in_progress, completed) or device serial number. Returns tasks sorted by creation time, newest first.

Show me all pending tasks

What tasks are associated with device 240488?

`t3000_task_update` — Update a task

Update a task's status, title, description, or priority. Use to mark tasks as in_progress or completed as you work through a workflow.

Mark task "Configure AHU-1" as completed

Change priority of task "Calibrate sensors" to high

`t3000_task_delete` — Delete a task

Delete a task by ID. Use to clean up completed or obsolete tasks.

Delete the completed commissioning tasks

Site Memory 3 tools

`t3000_memory_save` — Save site knowledge

Save a note about the building site for future reference. Memories persist across sessions and are automatically loaded into the AI context. Use for site-specific conventions, device naming patterns, or user preferences.

Remember that AHU-3 is the main unit and AHU-2 is decommissioned

Save a note: device naming convention is Building-Floor-Type-Number

`t3000_memory_list` — List site memories

List all saved site memories with optional filtering by category or text search. Returns memories sorted by last update time.

What do we know about this site?

Show me memories about AHU layout

`t3000_memory_delete` — Delete a memory

Delete a specific memory entry by key. Use to remove outdated or incorrect site information.

Forget the old AHU layout notes

Navigation 6 tools

`t3000_nav_list` — List all UI pages

List all pages in the T3000 web UI with paths, titles, keyboard shortcuts, and whether they require a device to be selected. Use to help users find the right page.

What pages are available in the T3000?

List all monitoring pages

`t3000_nav_search` — Search UI pages

Search for T3000 pages and topics by keyword. Returns matching pages ranked by relevance. Use when the user asks "How do I..." or "Where is..." questions about the T3000 interface.

Where do I configure PID loops?

How do I set up schedules?

`t3000_nav_redirect` — Navigate to a page

Get the URL to navigate to a specific T3000 page, optionally with a device pre-selected. The frontend uses this URL to redirect the user.

Open the outputs page for device 233626

Take me to the alarms page

`t3000_page_info` — Page details

Get detailed information about a T3000 page: what it does, what you can view/edit/configure, related MCP tools, keyboard shortcuts, and available features.

What can I do on the Alarms page?

Tell me about the PID Loops page

`t3000_device_current` — Current device context

Get the currently selected device in the web UI. Returns device type, name, and a note about its role. Use to discover which device the user is working with before querying points.

Which device am I currently working with?

What device is selected right now?

`t3000_set_chat_device` — Set the device to use for MCP operations

Confirm which device the AI should use for MCP operations. Call after the user confirms which device to work with (which may differ from the UI-selected device). This sets the authoritative chat device context.

Use device 233626 for the rest of this conversation

Switch the current device to 240488

Quick Test

After setup, try these two to confirm everything works:

List the Haystack auto-tagging rules

`haystack_list_rules`

Show me the input points for device T3-NB-ESP

`device_get_points`